Codebase Execution & Test Layouts

This reference guide documents the complete recursive script architecture of both the execution codebase (`src/`) and the comprehensive testing suite (`test_suite/`). It maps every file, sub-package, and test module to its core logical concern within our visual Multimodal RAG pipeline.

---

1. Codebase Execution Layout (`src/`)

```
src/
├── __init__.py                                   Central package
initialization namespace
├── hitl/                                         Human-In-The-
Loop (HITL) checkpoints and gates
│   ├── __init__.py                               HITL sub-
package initializer
│   ├── define_layers.py                          Ontology layer
definition seeder (Gate 1)
│   ├── documentation_layers.py                   Document
metadata layering manager (Gate 2)
│   ├── review.py                                 Console-based
terminal review gateway tool
│   └── gates/                                    Transition
verification gateways (Gate boundaries)
│       ├── __init__.py                           Gates sub-
package initializer
│       ├── common.py                             Shared human-
override helper routines
│       ├── gate2_doc_layers.py                   Document
approval ledger seeder
│       ├── gate3_regions.py                      Proposed
bounding box promotion ledgers (Gate 3)
│       └── gate4_assets.py                       Unified
metadata and layer asset approvals (Gate 4)
├── orchestration/                                Pipeline
sequencers, cli controllers, and scopes
│   ├── __init__.py                               Orchestration
sub-package initializer
│   ├── cli.py                                    Main user-
facing `rag` console entrypoint CLI
│   ├── init.py                                   Pre-flight
workspace scaffolding auditor
│   ├── orchestrator.py                           Core sequence
```

```
tracker and step sequencer
│   ├── sanity.py                                   Corpus and
schema integrity check controllers
│   ├── scope.py                                    Absolute
pathing and dynamic root resolver
│   ├── settlement.py                               Workspace
folder creation and marker settlement
│   ├── verify.py                                   Programmatic
catalog integrity verification (Gate 5)
│   ├── workers_defaults.py                         Global
concurrency and thread-pool parameters
│   ├── cli_phases/                                 Subcommand CLI
subcommand builders
│   │   ├── __init__.py                             CLI phase sub-
package initializer
│   │   ├── p01_scaffolding.py                      CLI controller
for checks, settlement, and reset
│   │   ├── p02_ingestion.py                        CLI controller
for seeding, layering, and ingestion
│   │   ├── p03_discovery.py                        CLI controller
for macro drafting and verification
│   │   ├── p04_curation.py                         CLI controller
for detail subdivisions and captioning
│   │   └── p05_indexing.py                         CLI controller
for index compilation, REPL, and exporting
│   └── phases/                                     Procedural
orchestrator phase executors
│       ├── __init__.py                             Orchestrator
phases sub-package initializer
│       ├── p01_scaffolding.py                      Pre-flight
environment and settlement runner
│       ├── p02_ingestion.py                        Ontology
layering and dual-stage document ingestion
│       ├── p03_discovery.py                        Multi-threaded
visual agents and programmatic sweep loops
│       ├── p04_curation.py                         Caption
rendering, vision parsing, and layer synchronization
│       └── p05_indexing.py                         FTS5 building,
vector index builders, and export compilers
├── pdf_crop/                                       Visual
intelligence, cropping, and correction engines
│   ├── __init__.py                                 Cropping sub-
package initializer
│   ├── agent_common.py                             Shared tool-
spec schemas and base64 pruners
│   ├── agent_defaults.py                           Global prompt
parameters and visual temperature scales
│   ├── agent_helpers.py                            Spatial
coordinate validation and rendering locks
```

```
│   ├── caption.py                                  Region
captioning and vision prompt generators
│   ├── crop.py                                     Multi-page PDF
rendering and memory coordinate crawlers
│   ├── crop_exceptions.py                          Visual engine
error classifications and brief status codes
│   ├── crop_helpers.py                             Adaptive
megapixel scaling and visible page boundary box clippers
│   ├── macro_handlers.py                           Structural
macro tool dispatchers and closures
│   ├── macro_images_agent.py                       Phase 1
drafting agent multi-turn master orchestrator
│   ├── macro_images_prompts.py                     Structural
macro grid and spatial prompt specifications
│   ├── micro_handlers.py                           Detail
subdivision tool dispatchers and closures
│   ├── micro_images_agent.py                       Phase 2
subdivision agent multi-turn master orchestrator
│   ├── micro_images_prompts.py                     Detail zoom and
micro-item prompt specifications
│   ├── recrop_agent.py                             Pro-tier multi-
turn visual re-crop refinement agent
│   ├── recrop_prompts.py                           Spatial
correction, grid overlays, and margin error prompts
│   ├── tool_defs.py                                Standard API
tool call schema descriptors
│   ├── verify_loop.py                              Verification
suite promoter, controller, and correcting sweeps
│   ├── agent_core/                                 Visual agent
session loaders and dispatchers
│   │   ├── __init__.py                             Agent core
initializer
│   │   ├── api_sender.py                           Single-turn LLM
message transport and exception handlers
│   │   ├── loop_orchestrator.py                    Multi-turn
agent conversation session drivers
│   │   ├── protocol_sync.py                        Strict
sequential Turn-Throttling guardrails
│   │   ├── sdk_helpers.py                          Unified model
client initialization and connection managers
│   │   ├── session_loader.py                       Disk-based
drafts loaders and cache coordinators
│   │   └── tool_dispatcher.py                      Interactive
client tool dispatching maps
│   └── verify_core/                                Programmatic
verification loop controllers
│       ├── __init__.py                             Verifier core
initializer
│       ├── promoter.py                             State
```

```
transition and candidate promotion drivers
│       ├── recrop_runner.py                          Subprocess
spawning for local re-crop agent refinements
│       ├── sdk_helpers.py                            One-shot
verification client transports
│       ├── tracker.py                                Thread-safe
verification metric and warning trackers
│       └── verifier.py                               Flash-tier
image verifiers and criteria checks
├── rag/                                              Retrieval-
Augmented Generation query and indexing engines
│   ├── __init__.py                                   RAG sub-package
initializer
│   ├── answer.py                                     Gemini-Flash
query synthesizer and citation engines
│   ├── asset_layers.py                               Asset
classification metadata loaders (Gate 4)
│   ├── census.py                                     Page and macro
view checklist file-system database
│   ├── chat.py                                       REPL
interactive chat loops and parameter managers
│   ├── conflicts.py                                  Synthesizer
citation contradiction capture database
│   ├── database.py                                   File-system
regions and metadata SQLite relational store
│   ├── export.py                                     Finalized
catalog export compilers and packaging
│   ├── index_build.py                                Multilingual
indices build coordinator and transaction rollbacks
│   ├── index_defaults.py                             Single source
of truth for indexing and vector chunk batch limits
│   ├── indexers/                                     Index compilers
for keyword and vector channels
│   │   ├── __init__.py                               Indexers
initializer
│   │   ├── captions_indexer.py                       Assets caption
indexer (ChromaDB channel)
│   │   ├── ids_indexer.py                            Assets display
identifier indexer (FTS5 channel)
│   │   ├── images_indexer.py                         Exploded image
tiles embedding indexer (ChromaDB channel)
│   │   ├── layers_indexer.py                         Ontology layer
context indexer (ChromaDB channel)
│   │   ├── text_cleaners.py                          Prose content
parser and non-text filters
│   │   └── text_indexer.py                           Page text chunk
embeddings indexer (ChromaDB/FTS5 channel)
│   ├── ingest.py                                     Dual-stage
document ingestion and seeder
```

```
│   ├── inventory.py                                    Single-source-
of-truth JSON/YAML catalogs and memory proxies
│   ├── json_repair.py                                  Resilient
brace-counting and quote-escaped parser repairs
│   ├── layer_index.py                                  Layer
descriptor affinity scoring database
│   ├── layers.py                                       Ontology layer
parsing and metadata validation
│   ├── max_tokens_defaults.py                          Model-specific
context parameters and maximum tokens
│   ├── models/                                         Model
interfaces and local ML modules
│   │   ├── __init__.py                                 Models re-
export namespace (100% backward-compatible)
│   │   ├── gemini_api.py                               Gemini remote
API client interfaces
│   │   ├── local_embeddings.py                         Local
PyTorch/OpenCLIP tile embedders
│   │   └── local_reranker.py                           Local Cross-
Encoder semantic rerankers
│   ├── page_fallback.py                                Multimodal raw
text extraction and on-the-fly visual page rendering
│   ├── plot_layers.py                                  Katona-standard
layered cyclical categorical plotting
│   ├── propose_urn.py                                  Deterministic
URN generation and slug conversion
│   ├── query.py                                        Standardized
single-turn query routing drivers
│   ├── query_expansion.py                              Multilingual
keyword rewrite and fusion-combining engine
│   ├── regions.py                                      Regions catalog
update routines and checkpoints
│   ├── retrieve.py                                     Reciprocal Rank
Fusion (RRF) blended multi-channel retrieval
│   ├── retrieve_defaults.py                            Default tuning
parameters, weights, and data structures for fanned-out RAG retrieval
│   ├── router.py                                       Topic-aware
keyword routing classifiers
│   ├── rules.py                                        Vision rules
database loader and parser
│   ├── scan_corpus.py                                  Recursive
directory crawler for PDF document ingestion
│   ├── text_patterns/                                  Multilingual
regex keyword parser models
│   │   ├── __init__.py                                 Patterns
initializer
│   │   ├── german.py                                   German language
block cleaners
│   │   ├── slovenian.py                                Slovenian
```

```
language block cleaners
│   │   └── universal.py                                Default
multilingual word pattern fallback matchers
│   ├── toc.py                                          Section-aware
hierarchy parsing and TOC extractors
│   ├── translate.py                                    Section-aware
build-time multilingual translations
│   └── violations.py                                   Formatting
violations Pydantic schemas and saving utilities
└── utils/                                              Operational
proxies, utilities, and safety tools
    ├── __init__.py                                     Utils package
initializer
    ├── census_stats.py                                 Page parsing
counts and metric analyzers
    ├── console.py                                      Single-instance
stdout/stderr consoles and safe_print
    ├── console_proxy.py                                Global mock-
like console wrapper proxy
    ├── DynamicPathProxy.py                             Import-time
path dynamic resolution proxy
    ├── index_commit.py                                 Stateless index
database chunk and batch commits
    ├── hf_quiet.py                                     HuggingFace
telemetry and rate-limit silencings
    ├── log_errors_parser.py                            JSON-Lines
logging callback parser
    ├── mupdf_warnings.py                               Symmetrical C-
level stderr warning suppression redirects
    ├── observe.py                                      Operational
latency and telemetry trackers
    ├── project_gitignore.py                            Permissive
workspace .gitignore builders
    ├── resilience.py                                   Exponential
backoffs and multi-tier exception briefers
    ├── resilience_defaults.py                          Network
timeout, retries, and backoff limits
    ├── sandbox_isolation.py                            Isolated test
mock and file-system path fixtures
    ├── slugs.py                                        Safe file-
system character conversions
    └── sweep.py                                        Automated file-
system scratch buffer purgers
```

2. Codebase Testing Layout (`test_suite/`)

```
test_suite/
├── __init__.py                                  Central testing
package initializer
├── conftest.py                                  Root-level
Pytest config, fixture hook setups, and Sandbox isolation
├── extract_samples.py                           Standalone CLI
utility to extract raw visual sandbox page samples
├── test_documentation_layers.py                 Offline
document classifier and layer tagging unit tests
├── test_preview_max_dim.py                      Offline 40-
megapixel resolution downscale limit tests
├── integration/                                 System
integration, database schemas, and retrieval tests
│   ├── test_cli_query_defaults.py               Verify
automatic search-weights routing defaults
│   ├── test_crop_render.py                      Coordinate-
clipping and crop rendering integrations
│   ├── test_handle_grid_overview.py             Page-dimension
grid overlay generation tests
│   ├── test_index_build_schema.py               FTS5 full-text
match schema integrations
│   ├── test_layer_index.py                      Ontology layer
descriptors affinity-reweighting tests
│   └── test_retrieve_machine_filter.py          Multi-channel
retrieval and machine-filtering tests
├── live/                                        Live, un-mocked
E2E integration and Visual Agent tests
│   ├── test_api_key_connectivity.py             Live Gemini
preflight and transport diagnostic tests
│   ├── test_batch_query.py                      Parallel
thread-pool bulk query matching live tests
│   ├── test_chat.py                             Interactive
REPL conversation turn-by-turn live tests
│   ├── test_images_agent_live.py                Live multi-turn
Macro/Micro visual drafting agents tests
│   ├── test_pipeline_e2e_live.py                Full-pipeline
bootstrap-to-repl un-mocked execution tests
│   ├── test_translate.py                        Live section-
aware translation API tests
│   ├── test_verify_integration.py               Live
programmatic verifier and correcter loop tests
│   └── test_visible_live_agent_tests.py         Live visible
coordinates grid-grounding agent tests
└── unit/                                        Swift, isolated
offline business logic unit tests
```

```
    ├── test_answer_prompt.py                          Prose citation
formatting and URN constraint tests
    ├── test_assessment_exceptions.py                  Unit tests for
the assessment exception propagation and skip-on-error routing
    ├── test_asset_layers.py                           Assets layer
matching rules and Gate 4 parsing tests
    ├── test_caption.py                                Region
captioning and vision description parser tests
    ├── test_catalog_schema.py                         Pydantic v2
metadata catalog validation tests
    ├── test_census_stats.py                           checklists-on-
disk count and stats calculation tests
    ├── test_conflicts.py                              Synthesizer
contradiction tracking database tests
    ├── test_console.py                                Thread-safe
parallel logging and ConsoleProxy tests
    ├── test_direct_page_fallback.py                   Direct page
text extraction and visual rendering fallback tests
    ├── test_documentation_sync.py                     Invariants,
rules, and documentation conformity tests
    ├── test_export.py                                 Finalized
catalog export JSON packaging tests
    ├── test_fts5_sanitize.py                          SQLite FTS5
keyword query cleaning and character tests
    ├── test_index_build_filter.py                     Prose-filtering
and table-exclusion tests
    ├── test_index_build_guard.py                      HITL Gate 2-4
approval bypass prevention checks tests
    ├── test_ingest_toc.py                             Page-by-page
document language classification tests
    ├── test_inventory_split.py                        Multi-project
concurrent workspace isolation tests
    ├── test_inventory_toc.py                          Table-of-
contents hierarchy section extractors tests
    ├── test_orchestrator_status.py                    Operational
sequence tracking and status code tests
    ├── test_programmatic_guards.py                    Pixel limits,
atomic writes, and file promotion tests
    ├── test_propose_urn.py                            URN provenance
formatting and slug conversion tests
    ├── test_query_expansion.py                        Multilingual
query rewrites and hybrid fusion tests
    ├── test_resilience.py                             Exponential
backoffs and multi-tier exception brief tests
    ├── test_resilient_parsing.py                      quote-escaped
brace-counting JSON repair tests
    ├── test_router.py                                 Topic-aware
query classification and channel-routing tests
    ├── test_scan_corpus.py                            Recursive PDF
```

```
document crawlers and filename parsing tests
    ├── test_text_patterns.py                          Multilingual
word boundary text clean regex tests
    ├── test_toc.py                                    Prose block
indexing and section segmentation tests
    ├── test_verifier_token_usage.py                   Flash-tier
judge token budget usage tracking tests
    ├── test_verify.py                                 Programmatic
integrity checks (Gate 5) verification tests
    └── test_violations.py                             Formatting
violations database schema and atomic write tests
```